OPENID CONNECT

OIDC Schulung

Internes Training für Entwickler

OIDC | OAuth 2.0 | PKCE | Keycloak | Angular SPA

Weiter mit Leertaste →

Agenda

TEIL 1
Grundlagen
- OIDC
- OAuth 2.0
- Tokens & Ablauf
- Scopes & Claims

TEIL 2
Authorization Code Flow
- Überblick & Schritte
- State & Nonce
- ↳ PKCE

TEIL 3
Integration
- SSO
- SPA Integration
- Keycloak Setup
- Konfiguration

Pause

TEIL 4
Hands-on
- OIDC Flow
- Demo

Mittags-pause

TEIL 5
Real World
- MeinAMS
- Keycloak Cluster
- Custom SPI

TEIL 1

OIDC Grundlagen

Was ist OpenID Connect?

OpenID Connect (OIDC) ist eine Identitätsschicht auf Basis von **OAuth 2.0**.

- Standardisiertes Protokoll zur **Authentifizierung**
- Aufgebaut auf **OAuth 2.0** (Autorisierung)
- Gibt Anwendungen die Möglichkeit, die **Identität eines Nutzers** zu verifizieren
- Verwaltet von der **OpenID Foundation**

OpenID Connect

Authentifizierung · ID Token · Wer ist der Nutzer?

↓ erweitert ↓

OAuth 2.0

Autorisierung · Access Token · Darf die App zugreifen?

Die wichtigsten Begriffe

Identity Provider (IdP)

Authentifiziert Nutzer und stellt Tokens aus. Beispiele: Keycloak, Azure AD, Google

Client

Die Anwendung (SPA oder Backend), die im Namen des Nutzers Tokens anfordert.

Resource Server

Der API-Server, der geschützte Ressourcen bereitstellt und Access Tokens validiert.

Tokens

ID Token (Nutzeridentität), Access Token (Zugriffsberechtigung), Refresh Token (erneuert Access Tokens)

Scope

Berechtigungsanfrage des Clients. Bestimmt, welche Claims im Token landen. z. B. `openid`, `profile`

Claims

Schlüssel-Wert-Paare im Token - Aussagen über den Nutzer. z. B. `sub`, `name`, `email`

Scopes & Claims

Scopes - was wird angefragt?

Scopes sind **Berechtigungsanfragen** des Clients. Der Nutzer stimmt ihnen beim Login zu.

Scope	Enthaltene Infos
`openid`	`sub` (Pflicht für OIDC)
`profile`	`name`, `given_name`,...
`email`	`email`

`scope=openid profile email`

Claims - was steht im Token?

Claims sind die **tatsächlichen Daten** im JWT-Payload. Ein Scope bestimmt, welche Claims ausgestellt werden.

```
{
  "sub": "user-abc123",        // openid
  "name": "Max Mustermann",    // profile
  "email": "max@example.com",  // email
  "roles": ["admin"]           // custom
}
```

OAuth 2.0 vs. OpenID Connect

OAUTH 2.0

Autorisierung

"Die App darf auf meine Google Drive Dateien zugreifen."

- → Ergebnis: **Access Token**
- → Berechtigung für API-Zugriff
- → Nutzeridentität: **unbekannt**

```
scope=drive.readonly
→ Access Token
```

OPENID CONNECT

Authentifizierung + Autorisierung

"Die App hat Informationen über meine Persönlichen Daten."

- → Ergebnis: Access Token + **ID Token**
- → ID Token enthält **Nutzeridentität**
- → Nutzerinfos können auch direkt in den **Access Token** gemappt werden
- → Scope `openid` aktiviert OIDC

```
scope=openid profile email
→ Access Token + ID Token
```

Tokens im Überblick

ID Token

JWT mit **Nutzeridentität**

→ Für die **Client-App**

Enthält: `sub`, `name`, `email`, `exp`

✗ Nie an APIs weitergeben

Access Token

JWT mit **Berechtigungen**

→ Für den **Resource Server**

Enthält: `scope`, `roles`, `exp`

Kurzlebig (Minuten)

Refresh Token

Erneuert **Access Tokens**

→ Kein erneuter Login nötig

Opaker String (kein JWT)

⚠ Sicher speichern!

JSON Web Token (JWT)

Aufbau: 3 Base64url-Segmente

```
Header: eyJhbGciOiJSUzI1NiJ9
Payload: eyJzdWIiOiJ1c2VyLTEyMyJ9
Sig: SflKxwRJSMeKKF2QT4fw...
```

Header - Algorithmus & Token-Typ

```
{ "alg": "RS256", "typ": "JWT" }
```

Payload - Claims (die eigentlichen Daten)

```
{ "sub": "user-123", "name": "Max", "exp": 1710000000 }
```

Signatur - Integrität & Herkunft

```
RS256( base64url(header) + "." + base64url(payload), privat
```

Validierung ohne Roundtrip

Der Resource Server prüft die Signatur mit dem **Public Key des IdP** (via JWKS Endpoint).

JWKS Endpoint muss nur **einmal** aufgerufen werden!

⚠ JWTs sind nur **Base64url-kodiert**, nicht verschlüsselt.
Der Payload ist für jeden lesbar → **keine Secrets im Token!**

→ Dekodieren & prüfen: **jwt.io**

TEIL 2

Authorization Code Flow

Authorization Code Flow - Überblick

Der **Authorization Code Flow** ist der empfohlene OAuth 2.0 Flow für alle Clients, die einen Nutzer authentifizieren müssen.

Die App bekommt nie direkt ein Token, stattdessen erst einen kurzlebigen **Authorization Code**, der dann gegen Tokens eingetauscht wird.

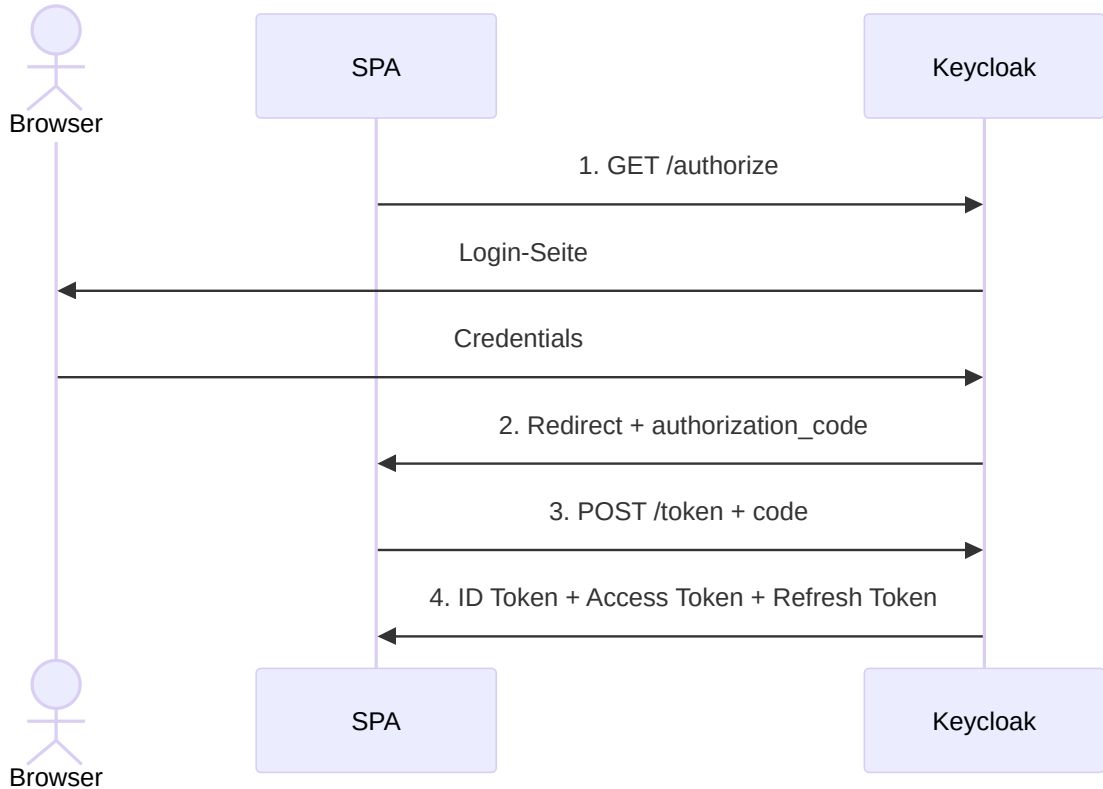
Geeignet für

SPAs, Web Apps, Native Apps, Backend-to-Backend

Nicht mehr empfohlen

Implicit Flow (Token direkt im Redirect), Resource Owner Password Flow

Authorization Code Flow - Die 4 Schritte



State & Nonce

`state` - CSRF-Schutz

- Zufälliger Wert, den die SPA vor dem Redirect generiert
- Wird mit dem Authorization Request mitgeschickt
- Kommt im Callback **unverändert zurück**
- SPA prüft ob der Wert übereinstimmt

```
GET /authorize?...&state=xK3p9zQ2
→ callback?code=...&state=xK3p9zQ2 ✓
```

Abweichender `state` = möglicher CSRF-Angriff → Request verwerfen

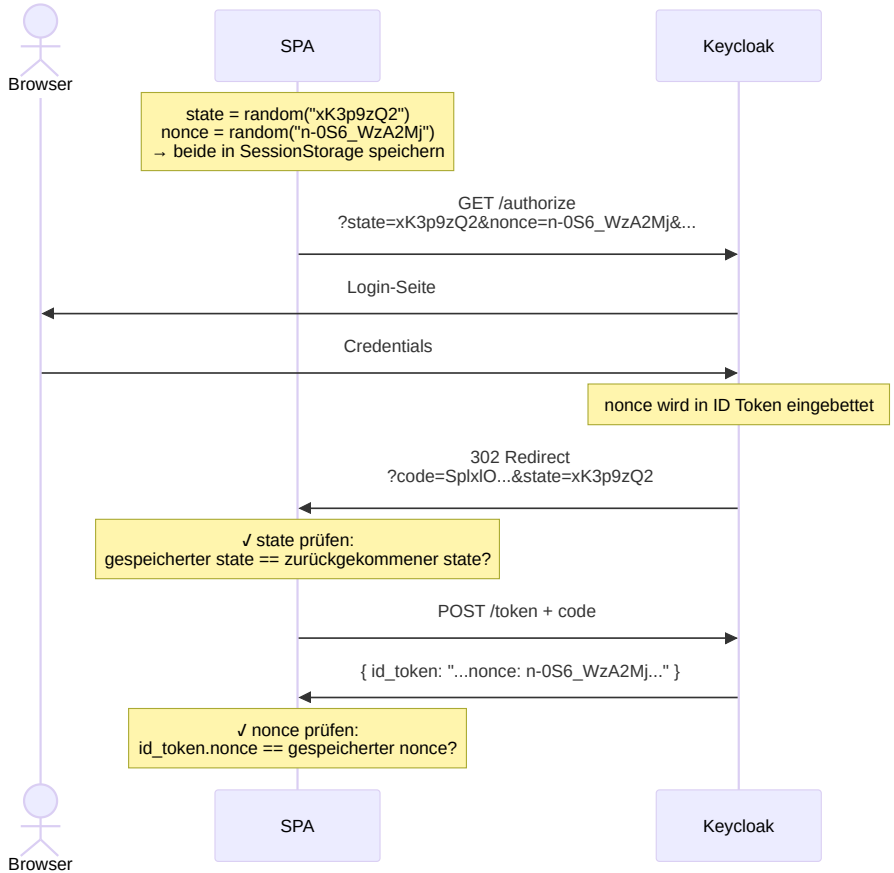
`nonce` - Replay-Schutz

- Zufälliger Wert, der im **ID Token** als Claim landet
- SPA prüft: nonce im Token == gesendeter nonce?
- Verhindert, dass ein altes Token erneut eingespielt wird

```
// ID Token Payload
{ "nonce": "n-0S6_WzA2Mj", ... }
```

Beide Parameter sollten kryptografisch zufällig sein - keine vorhersehbaren Werte!

State & Nonce - Ablauf im Flow



Warum PKCE?

Das Problem bei SPAs & Native Apps

- **Kein sicheres Client Secret** möglich
  - JavaScript-Code ist öffentlich einsehbar
  - Native Apps können inspiziert werden
- Authorization Code kann **abgefangen** werden
  - Malicious Apps mit gleichem Redirect URI
  - Code Injection in Redirects

Die Lösung: PKCE

Proof Key for Code Exchange (RFC 7636)

- Kein Secret nötig
- Client erzeugt **einmaliges Schlüsselpaar** pro Flow
- Code Verifier (geheim) + Code Challenge (öffentlich)
- Authorization Code ist **an die Challenge gebunden**
- Angreifer ohne Verifier kann Code nicht nutzen

PKCE: Code Verifier & Code Challenge

Code Verifier

- Zufälliger String (43-128 Zeichen)
- Nur Base64url-Zeichen: `[A-Z a-z 0-9 - . _ ~]`
- Wird **nur im Client** gespeichert (Memory!)
- Niemals übertragen beim Authorization Request

```
const verifier = base64url(
  crypto.randomBytes(32)
)
```

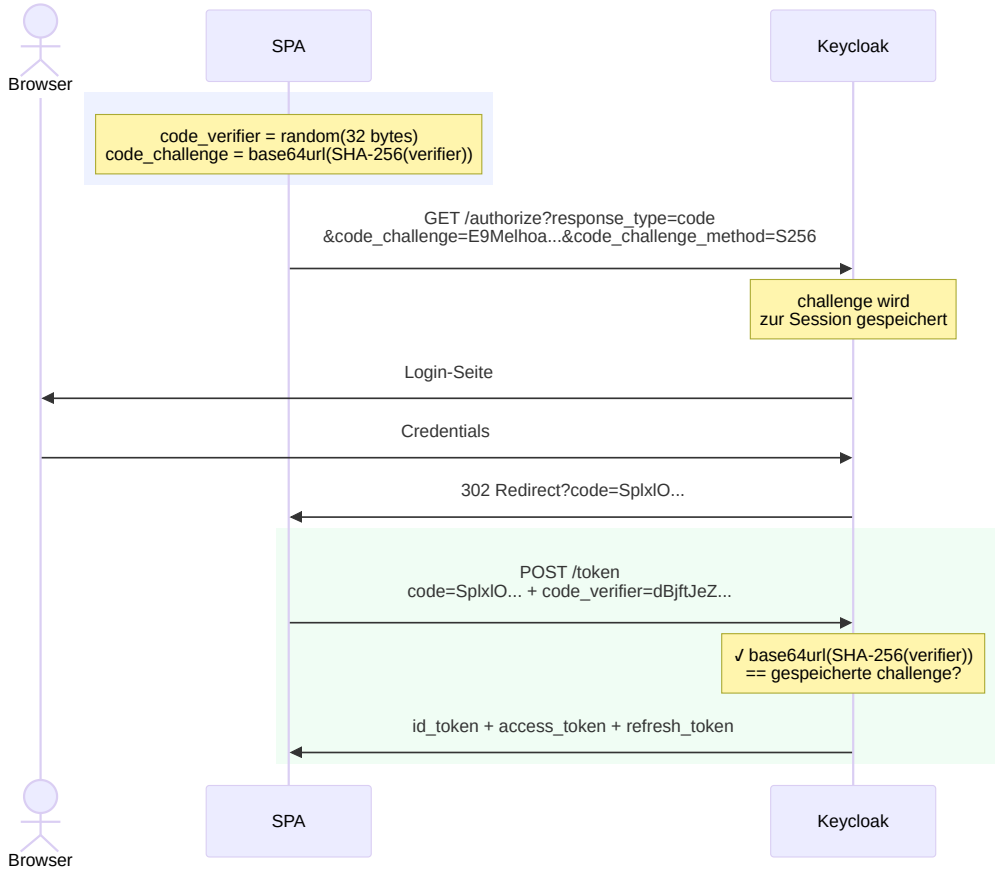
Code Challenge

- Abgeleitet aus dem Code Verifier
- Methode `S256` (SHA-256, empfohlen) oder `plain`

```
const challenge = base64url(
  sha256(verifier)
)
```

- Wird **mit dem Authorization Request** gesendet
- IdP speichert sie zum späteren Vergleich

PKCE Flow: Schritt für Schritt



Schritt 1: Authorization Request

SPA leitet den Browser zum IdP weiter.

```
GET /realms/demo/protocol/openid-connect/auth HTTP/1.1
Host: keycloak.example.com

  ?response_type=code
  &client_id=my-spa
  &redirect_uri=https://app.example.com/callback
  &scope=openid profile email
  &state=xK3p9zQ2
  &nonce=n-0S6_WzA2Mj
  &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
  &code_challenge_method=S256
```

- `scope=openid` - aktiviert OIDC (ohne diesen Parameter: reines OAuth 2.0)
- **state**: zufälliger Wert, kommt in Schritt 2 zurück (CSRF-Schutz)
- **code_challenge**: `SHA-256(code_verifier)`, wird in Schritt 3 geprüft
- `code_challenge_method=S256`: gibt den Hash-Algorithmus an

Schritt 2: Authorization Response

Nach erfolgreichem Login leitet der IdP zurück zur App.

```
HTTP/1.1 302 Found
Location: https://app.example.com/callback
  ?code=SplxlOBeZQQYbYS6WxSbIA
  &state=xK3p9zQ2
```

- **code**: kurzlebiger Authorization Code (~60 Sek., einmalig)
- Noch **kein Token**
- **state**: muss mit Schritt 1 übereinstimmen (CSRF-Check)
- Bei Abweichung: **Request verwerfen**

Schritt 3: Token Request

SPA tauscht den Code gegen Tokens ein.

```
POST /realms/demo/protocol/openid-connect/token HTTP/1.1
Host: keycloak.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=SplxlOBeZQQYbYS6WxSbIA
&redirect_uri=https://app.example.com/callback
&client_id=my-spa
&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

- **code**: derselbe Authorization Code aus Schritt 2
- **code_verifier**: das Original-Secret
- IdP prüft: **code_challenge** aus Schritt 1 ✓
- Bei Mismatch: **Token wird verweigert**

Schritt 4: Token Response

IdP antwortet mit drei Tokens.

```
{
  "id_token":      "eyJhbGciOiJSUzI1NiJ9...",
  "access_token":  "eyJhbGciOiJSUzI1NiJ9...",
  "token_type":    "Bearer",
  "refresh_token": "8xLOxBtZp8vRHXkgHQu...",
  "expires_in":    300
}
```

ID Token: JWT für die App Enthält: `sub`, `name`, `email`

Access Token: für APIs Enthält: `scope`, `roles`, `exp`

Refresh Token Erneuert Access Tokens

```
// ID Token (dekodiert)
{
  "sub":   "user-abc123",
  "email": "max@example.com",
  "iss":   "https://keycloak/realms/demo",
  "aud":   "my-spa",
  "exp":   1710000000,
  "nonce": "n-0S6_WzA2Mj"
}
```

→ **nonce** aus Schritt 1 muss mit dem gespeicherten Wert übereinstimmen

Sicherheitsmechanismen im Überblick

state

CSRF-Schutz

Zufälliger Wert im Authorization Request - muss in der Response identisch zurückkommen.

Bei Abweichung → Request verwerfen.

Schritt 1 → Schritt 2

nonce

Replay-Schutz

Zufälliger Wert im Authorization Request - taucht als Claim im ID Token auf.

Bei Abweichung → Token verwerfen.

Schritt 1 → Schritt 4

code_challenge

Code-Theft-Schutz

SHA-256 des code_verifier - gebunden an den Authorization Code.

Schritt 1 → Schritt 3

TEIL 3

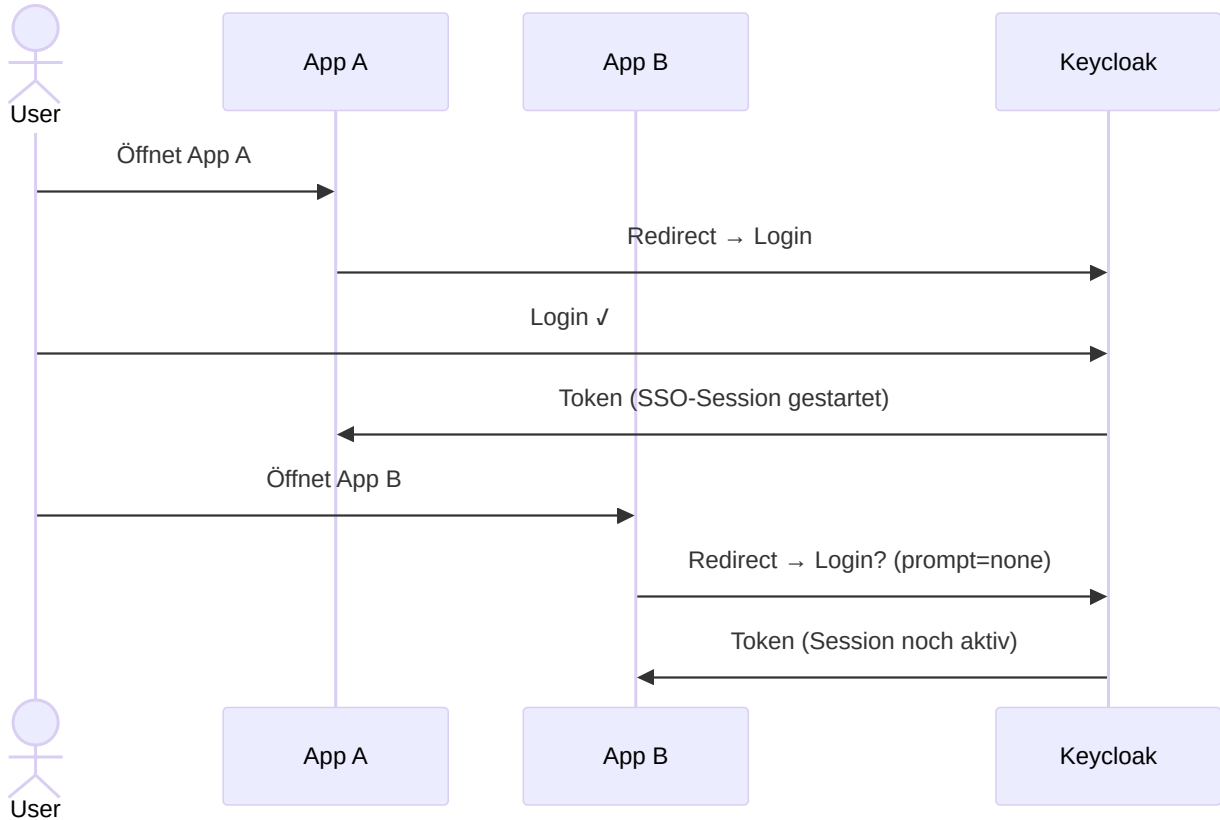
Implementierung

Single Sign-On (SSO)

Der Nutzer authentifiziert sich **einmal beim IdP** und ist damit automatisch in allen verbundenen Apps eingeloggt.

- Keycloak speichert nach dem Login eine **SSO-Session** (serverseitig + Cookie im Browser)
- Öffnet der Nutzer eine zweite App, erkennt Keycloak die Session → **kein erneuter Login nötig**
- Logout muss **serverseitig** beim IdP passieren

Single Sign-On (SSO): Ablauf



Einbindung in eine SPA

Installation

```
npm install angular-auth-oidc-client
```

app.config.ts

```
import { PassedInitialConfig } from 'angular-auth-oidc-client';

export const authConfig: PassedInitialConfig = {
  config: {
              authority: 'http://localhost:8080/realms/test-realm',
              redirectUrl: window.location.origin + '/callback',
              postLoginRoute: '/dashboard',
              postLogoutRedirectUri: window.location.origin,
              clientId: 'test-application',
              scope: 'openid profile',
              responseType: 'code',
              silentRenew: true,
              useRefreshToken: true,
              renewTimeBeforeTokenExpiresInSeconds: 30,
              autoUserInfo: true,
          }
}
```

Login & Token verwenden

auth.service.ts

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private oidc = inject(OidcSecurityService)

  login() {
    this.oidc.authorize()
  }

  logout() {
    this.oidc.logoff().subscribe()
  }

  accessToken$ = this.oidc.getAccessToken()
  userData$    = this.oidc.userData$
}
```

http-interceptor.ts

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  private oidc = inject(OidcSecurityService)

  intercept(req: HttpRequest<any>, next: HttpHandler) {
    return this.oidc.getAccessToken().pipe(
      take(1),
      switchMap(token =>
        next.handle(req.clone({
          setHeaders: { Authorization: `Bearer ${token}` }
        }))
      )
    )
  }
}
```

Token-Speicherung in SPAs

✕ Nicht empfohlen

localStorage / sessionStorage

- Zugänglich über JavaScript
- Anfällig für **XSS-Angriffe**
- Kein automatischer Ablauf

Cookies ohne HttpOnly

- Ebenfalls JS-zugänglich

✓ Empfohlen

Memory (JavaScript Variable)

- Kein Zugriff aus anderen Kontexten
- Erfordert Redirect bei Page Reload

HttpOnly + Secure Cookie (BFF-Pattern/Token Mediating Backend)

- Kein JS-Zugriff möglich
- Schutz vor XSS
- CSRF Schutz notwendig

RFC Draft

OAuth 2.0 for Browser-Based Applications

Token Refresh Flow

Warum Refresh Tokens?

Access Tokens sind **kurzlebig** (typisch 5 Minuten). Mit einem Refresh Token kann die App im Hintergrund neue Access Tokens holen, ohne den Nutzer erneut einzuloggen.

```
POST /realms/demo/protocol/openid-connect/token

grant_type=refresh_token
&refresh_token=8xLOxBtZp8vRHXkgHQu...
&client_id=my-spa
```

→ Keycloak antwortet mit einem **neuen Access Token** (und meist einem neuen Refresh Token, falls token rotation aktiv ist)

In angular-auth-oidc-client

```
silentRenew: true,
useRefreshToken: true,
renewTimeBeforeTokenExpiresInSeconds: 30
```

✅ **Refresh Token** (`useRefreshToken: true`)
HTTP POST im Hintergrund

⚠ **Silent Renew via iframe** (Fallback)
Neuer OIDC Flow im Hidden iframe - Abgelöst durch refresh Token

Token-Validierung im Backend

Was wird geprüft?

- `iss` : Issuer (Keycloak URL + Realm)
- `aud` : Audience (Client ID oder API)
- `exp` : Ablaufzeit
- `nbf` : Not Before
- **Signatur**: via JWKS Public Key

JWKS Endpoint

```
GET {issuer}/protocol/openid-connect/certs
```

Keys werden gecacht und automatisch rotiert.

Beispiel: Spring Boot

```java
@Configuration
public class SecurityConfig {

  @Bean
  SecurityFilterChain filterChain(HttpSecurity http) {
    http
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/public/**").permitAll()
        .anyRequest().authenticated()
      )
      .oauth2ResourceServer(oauth2 -> oauth2
        .jwt(jwt -> jwt
          .issuerUri("https://keycloak/realms/myrealm")
        )
      );
    return http.build();
  }
}
```

Logout: Client-Initiated Logout

✗ Local-only Logout

Token aus Memory löschen - **IdP-Session bleibt aktiv**.

→ Nächster Login-Versuch via Silent SSO: User ist sofort wieder eingeloggt.

✓ Client-Initiated Logout

Client redirectet den Browser zum `end_session_endpoint` - Keycloak zerstört die SSO-Session serverseitig.

```
// angular-auth-oidc-client
this.oidc.logoff().subscribe()
```

```
GET /realms/demo/protocol/openid-connect/logout
Host: keycloak.example.com

  ?id_token_hint=eyJhbGciOiJSUzI1NiJ9...
  &post_logout_redirect_uri=https://app.example.com
  &client_id=my-spa
```

→ **id_token_hint** - identifiziert die Session beim IdP

→ **post_logout_redirect_uri** - Ziel nach erfolgreichem Logout

→ URI muss in Keycloak als erlaubt registriert sein

Logout: Front-Channel vs. Back-Channel

Szenario: User loggt sich in App A aus. Wie erfahren App B und App C davon?

Front-Channel

via Browser · unsichtbare iframes

```
<iframe src="https://app-b/logout" />
<iframe src="https://app-c/logout" />
<iframe src="https://app-d/logout" />
```

⚠ Wird von modernen Browsern zunehmend blockiert

Back-Channel

server-to-server · HTTP POST

```
POST /logout
Body: logout_token (signiertes JWT)
```

✅ Zuverlässig, aber benötigt ein Backend

Logout: Implementierung im Backend

1. Keycloak konfigurieren

Admin Console → Client → **Advanced**:

```
Back-Channel Logout URL:
https://api.example.com/logout/backchannel
```

2. Spring Security konfigurieren

```java
@Bean
SecurityFilterChain filterChain(HttpSecurity http) {
  http
    .oauth2ResourceServer(oauth2 -> oauth2
      .jwt(Customizer.withDefaults())
    )
    .oidcLogout(logout -> logout
      .backChannel(Customizer.withDefaults())
    );
  return http.build();
}
```

3. Was Spring im Hintergrund tut

Spring Security registriert automatisch den Endpoint `POST /logout/connect/back-channel/{registrationId}` und:

- Validiert die Signatur des `logout_token`
- Prüft `iss`, `aud`, `iat`, `jti`
- Invalidiert die verknüpfte Session via `sub` oder `sid`

Keycloak als Identity Provider

Was ist Keycloak?

- Open-Source **Identity & Access Management**
- Von Red Hat entwickelt
- Unterstützt **OIDC**, OAuth 2.0, SAML 2.0
- Eigener **Realm** pro Mandant/Projekt
- Integrierter **Admin-Console** und **Account-Portal**

Features

- Social Login (Google, GitHub, …)
- Multi-Factor Authentication
- User Federation (LDAP / Active Directory)
- Fine-grained Authorization

Wichtige Keycloak-Konzepte

Begriff	Bedeutung
Realm	Mandant / Namespace
Client	Registrierte Anwendung
User	Endnutzer
Role	Berechtigungsgruppe
Scope	Zugriffsbereich
Flow	Auth-Ablauf (konfigurierbar)

Keycloak: Client-Konfiguration

Client anlegen (SPA)

1. **Client ID:** `my-spa`
2. **Client Type:** `Public` ← wichtig für SPA
3. **Valid Redirect URIs:** `https://app/*`
4. **Web Origins:** `https://app` (CORS)
5. **Standard Flow:** aktiviert
6. **Implicit Flow:** deaktiviert

Discovery Endpoint

```
GET https://keycloak/realms/{realm}
        /.well-known/openid-configuration
```

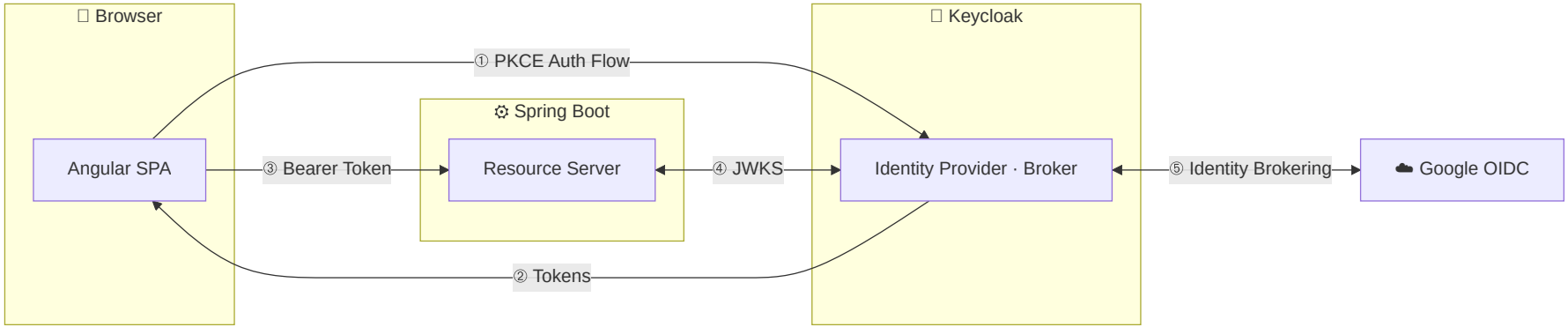
Liefert alle Endpoints automatisch:

```
{
  "issuer": "https://keycloak/realms/myrealm",
  "authorization_endpoint": ".../auth",
  "token_endpoint": ".../token",
  "userinfo_endpoint": ".../userinfo",
  "jwks_uri": ".../certs",
  ...
}
```

TEIL 4

Hands-on: OIDC Flow

Architekturübersicht



Setup

```
git clone <repo-url>
cd oidc-schulung

docker compose up -d
```

:4200 **Frontend** - Angular SPA
http://localhost:4200

:8090 **Backend** - Spring Boot API
http://localhost:8090

:8080 **Keycloak** - Admin Console
http://localhost:8080 · admin / admin

Zielbild: Keycloak Konfiguration

Realm

`test-realm`

Ein **Realm** ist ein isolierter Mandant in Keycloak. Jedes Realm hat eigene User, Clients, Rollen und Identity Provider.

Client

`test-application`

Ein **Client** repräsentiert die App, die Tokens anfordert.

Identity Provider

`Google`

Keycloak fungiert als **Broker** zu Google. Die App bekommt immer Keycloak-Tokens, egal wo der User sich anmeldet.

Identity Brokering

Warum Keycloak als Broker?

Die App bindet sich **einmalig an Keycloak**, nicht an jeden externen Provider. Keycloak übernimmt die Komplexität dahinter.

Token-Übersetzung
Google-Token bleibt intern. Die App bekommt immer Keycloak-Tokens

Claim-Mapping
Google-Claims (email, name, …) werden auf Keycloak-User gemappt

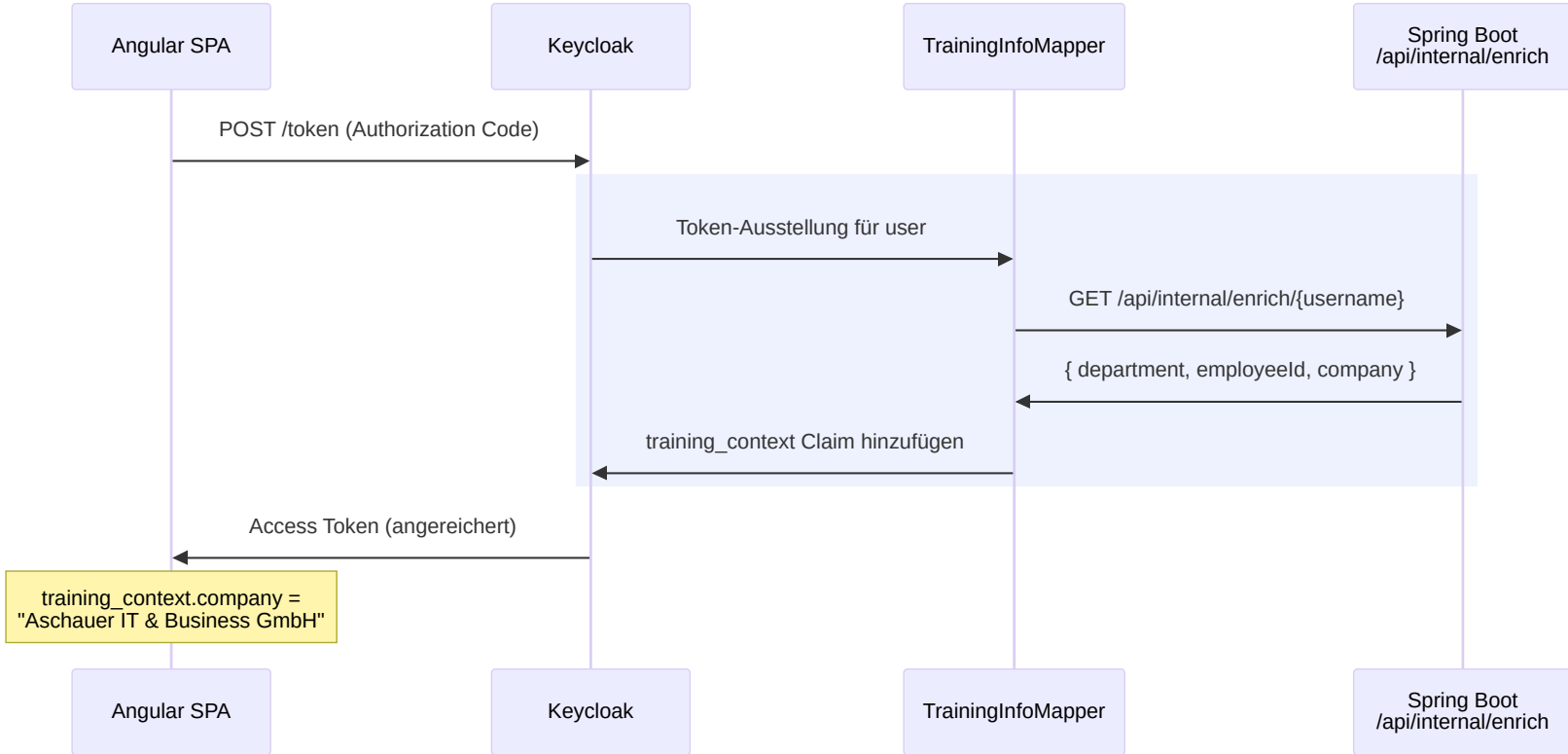
Austauschbar
Mehrere Identity Provider möglich, Google kann ausgetauscht werden ohne die App ändern zu müssen

Login-Flow mit Brokering

```
Browser          Keycloak         Google
  |                  |                |
  |── Login ────────▶|                |
  |                  |── Auth Req ──▶|
  |◀─ Redirect ──────|                |
  |── Google Login ──────────────────▶|
  |◀──────────────── Google Code ─────|
  |── Code ─────────▶|                |
  |                  |── Token Req ─▶|
  |                  |◀── Tokens ────|
  |◀─Keycloak Token─|                |
```

Keycloak tauscht den Google-Authorization-Code selbst gegen Tokens ein

Custom Protocol Mapper: Token-Anreicherung



TEIL 5

Real World Example

MeinAMS für Personen: Überblick

RAHMENBEDINGUNGEN

Benutzer	**~400.000**
Login	**ID-Austria · Username + Passwort**
Identity Provider	**Keycloak**

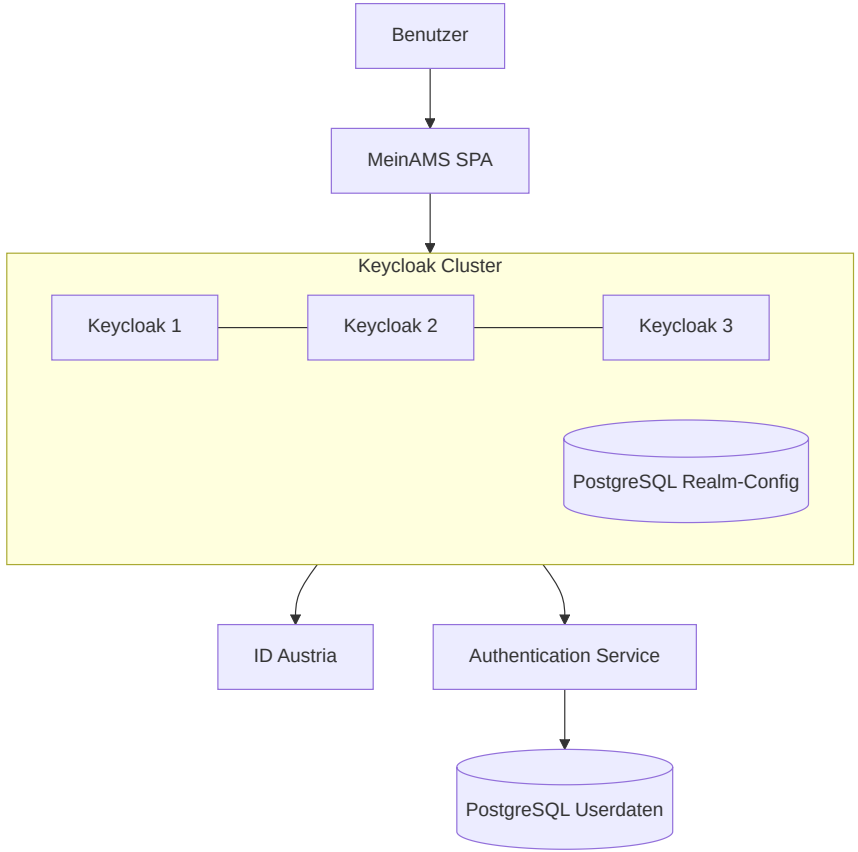
HERAUSFORDERUNGEN

Skalierbarkeit
Hohe Nutzerzahl, kein Single Point of Failure

Eigener User Store
Benutzerdaten getrennt von Keycloak - Custom SPI zur Anbindung

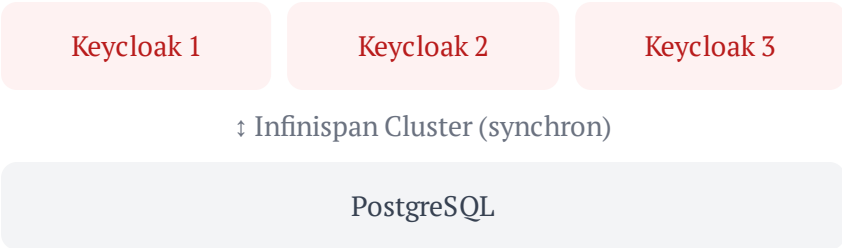
Custom Authentication Provider
Eigener Authenticator für die Passwortprüfung gegen den externen Store

MeinAMS: Architektur



MeinAMS: Keycloak Cluster & Performance

3 Keycloak-Instanzen



PostgreSQL

- Persistenz für **User, Clients, Realm-Konfiguration**
- Sessions leben im **Infinispan-Cache** (In-Memory)

Infinispan: Sync vs. Async

Synchrone Replikation (gewählt)

- Session-Daten sofort auf allen 3 Nodes sichtbar
- Kein Sticky-Session-Routing am Load Balancer nötig
- Bei Node-Ausfall: keine Sessions verloren

⚠ **Tradeoff:** Jeder Login wartet auf Bestätigung aller Nodes → erhöhte Latenz

Async wäre schneller, aber:

- Kurzes Zeitfenster wo Nodes inkonsistent sind
- Bei Failover in diesem Fenster → Session verloren

MeinAMS: Custom Authentication Service

Warum ein eigener Service?

Keycloak bietet zwar eine eigene User-Datenbank - aber die Integration einer **bestehenden User-DB** in Keycloak ist komplex und bindet die Datenhaltung an den IdP.

Ziele:

- User-Datenbank bleibt in **eigenem Spring Boot Service** - unabhängig von Keycloak
- Password Hashing **separat skalierbar** (CPU-intensiv bei 400k Usern)
- Vertraute Technologie statt Keycloak-interne User Federation

MeinAMS: Custom Authenticator

Ablauf

1. User gibt Passwort ein
2. Keycloak ruft den **Custom Authenticator** auf
3. Authenticator sendet Credentials per HTTP an den Authentication Service
4. Spring Boot Service prüft gegen die eigene PostgreSQL
5. Bei Erfolg: Keycloak stellt Tokens aus

Keycloak SPI: Custom Authenticator

```
public class CustomAuthenticator
    implements Authenticator {

  @Override
  public void authenticate(AuthenticationFlowContext ctx) {
    String username = getUsername(ctx);
    String password = getPassword(ctx);

    boolean valid = authService.verify(username, password);

    if (valid) {
      ctx.success();
    } else {
      ctx.failure(AuthenticationFlowError.INVALID_CREDENTIA
    }
  }
}
```

→ In Keycloak als **Authentication Flow** registriert

Fragen?

Danke fürs Mitmachen!

RFC 6749 - OAuth 2.0

RFC 7636 - PKCE

OpenID Connect Core 1.0

jwt.io · Keycloak Docs · angular-auth-oidc-client Docs

`github.com/philippcou/oidc-schulung-asc`